Listing 8.11. Mini-batch training using experience replay

```
def minibatch_train(use_extrinsic=True):
    state1_batch, action_batch, reward_batch, state2_batch =
replay.get_batch()
    action_batch = action_batch.view(action_batch.shape[0],1)
1
    reward_batch = reward_batch.view(reward_batch.shape[0],1)

    forward_pred_err, inverse_pred_err = ICM(state1_batch,
action_batch, state2_batch)
2
    i_reward = (1. / params['eta']) * forward_pred_err
3
    reward = i_reward.detach()
4
    if use_explicit:
5
        reward += reward_batch
    qvals = Qmodel(state2_batch)
6
    reward += params['gamma'] * torch.max(qvals)
    reward_pred = Qmodel(state1_batch)
    reward_target = reward_pred.clone()
    indices = torch.stack( (torch.arange(action_batch.shape[0]), \
7
    action_batch.squeeze()), dim=0)
    indices = indices.tolist()
    reward_target[indices] = reward.squeeze()
    q_loss = 1e5 * qloss(F.normalize(reward_pred), \
    F.normalize(reward_target.detach()))
    return forward_pred_err, inverse_pred_err, q_loss
```

- ***1*** We reshape these tensors to add a single dimension to be compatible with the models.
- ***2*** Runs the ICM
- ***3*** Scales the forward-prediction error using the eta parameter
- ***4*** Starts totaling up the reward; makes sure to detach the i_reward tensor
- ***5*** The use_explicit Boolean variable lets us decide whether or not to use explicit rewards in addition to the intrinsic reward.
- ***6*** Computes the action values for the next state
- ***7*** Since the action_batch is a tensor of integers of action indices, we convert this to a tensor of one-hot encoded vectors.